Rescue Router: few-shot flood mapping and rescue routing with Chameleon

Marlen Melis 20260876 KAIST AIC200 mini-project **Track A + Track B**

Summary. We adapt Chameleon to map flooded roads and buildings in drone photos from **50 labelled photos** (Track A) and build a rescue router, a web viewer and a Gradio app on its predicted maps (Track B). On the full 448-photo FloodNet test set the final model reaches **mIoU .507** [.474, .538] (5 classes, 95 % bootstrap CI; +.035 [+.013, +.057] over our first model after a pre-registered round of 36 improvement ideas); the router finds the houses that are cut off by road with **F1 .752** [.629, .851]. A fully supervised PSPNet trained on 1,445 labels reaches .848 on the same classes, so this is a fast first map, not a replacement for full labelling.

1 Problem and motivation

On 8 Aug 2022 Seoul recorded 141.5 mm of rain in one hour, the most since records began in 1907, and streets in Gangnam turned into rivers [6]. On 15 Jul 2023 an embankment failure flooded the Gungpyeong 2 underpass in Osong within minutes: 17 vehicles were trapped and 14 people died [7]. In the first hours of such an event, rescue coordinators need one answer per street and per house: *can a truck get there, does it need a boat, or is it cut off?* Drones deliver photos within the hour, but turning photos into a map takes people: pixel labels for the two public post-hurricane datasets took about **one hour per photo** [2, 3], and a supervised segmentation model needs the 1,445 labelled photos of FloodNet's training split. Our question: how good is a map adapted from a few labelled photos of the *new* disaster (50 photos ≈ 50 labeller-hours, 3.5 % of FloodNet's training labels), and is it good enough to route rescue teams, if the system says how much to trust it?

2 Formulation as structured prediction

Each 768×576 photo gets a **dense label with $C = 5$ channels** (right). The label is a one-hot $Y \in \{0,1\}^{5\times H\times W}$; pixels of every other class (tree, grass, vehicle, pool, background) are 0 in all five channels and are decoded as *other*; the mask M is 1 except on unlabelled pixels. Chameleon predicts one sigmoid map per channel (binary cross-entropy). A pixel takes the channel with the largest p_c/τ_c among those with $p_c \geq \tau_c$, else *other*; $\tau_c = 0.5$, or per-channel values tuned on 30 validation photos.

ch	FloodNet / RescueNet class	role in routing
0	road clear	truck
1	road flooded / road blocked	boat, no truck
2	building flooded / intact	target, obstacle
3	building dry / damaged	target, obstacle
4	water	boat

The channels are chosen for the consumer, a **router** (Track B). Its input is the cell grid of the decoded map (4×4-pixel cells) plus the water probability $p_{\text{wet}} = p_1 + p_4$; its output is a status per house (TRUCK, BOAT, FOOT, ISOLATED) and a route (§4). Inference tiles the photo with 384-px tiles at stride 192 (6 tiles) and blends them with Gaussian weights. **Rejected alternatives:** all 10 FloodNet classes (spends 50 shots on trees and grass the router does not need); a single "passable" channel (no targets, no boat leg); regressing a cost map (no labels exist); trees as obstacles (canopy hides real roads from above); pools as water (pools are not routes).

3 Data and resources

FloodNet [2] (Hurricane Harvey, Texas 2017; 2,343 drone photos, 4000×3000, split 1,445 / 450 / 448; CDLA-Permissive-1.0, https://github.com/BinaLab/FloodNet-Supervised_v1.0). **RescueNet** [3] (Hurricane Michael, Florida 2018; 4,494 photos; CC0 on figshare, doi:10.6084/m9.figshare.c.6647354; road blocked = blocked by water, sand or debris; building intact = no or minor damage). **We labelled nothing ourselves**: all labels are the datasets'. Photos were area-downsampled to 768×576 (labels nearest-neighbour; 57 FloodNet photos needed their EXIF 180° rotation applied). Splits were fixed before choosing supports and are checked by SHA-256 in the notebook: **supports** = 50 FloodNet training photos, chosen stratified so the rare flooded classes appear, and nested (the k-shot set is the first k); RescueNet supports come from its validation split. **val30** (30 val photos) selects checkpoints, decodes and router gates. **eval100** (100 test photos, half with flooding) gives the learning curve; the **full test** (448) gives the headline. We checked for test photos that show the same ground as a support photo (ORB + RANSAC): 14 of 448 do; removing them changes mIoU by ≤.004.

Pretrained weights. (1) The **Chameleon** meta-trained generalist checkpoint [1] distributed by the course (https://github.com/GitGyun/AIC200-Project-StructuredPrediction, shared Drive folder; model code https://github.com/GitGyun/chameleon). It contains (2) its BEiT-v2-Large image encoder [5] and (3) its ViT-L label encoder and DPT-style decoder as released by the Chameleon authors. (4) **SAM ViT-B** [4] (https://github.com/facebookresearch/segment-anything, Apache-2.0), used only to redraw the outline of each building Chameleon predicts (§4); every class comes from Chameleon. **Compute:** 16-GB NVIDIA T4s on Kaggle (and Colab), about 29 GPU-hours in total (12 of them for the improvement round).

4 Method and experimental setup

Fine-tuning (the same recipe for every run, sized so a 50-shot run fits a T4; right). Only the parameters the starter fine-tunes are trained (biases, the matching temperature, a LayerNorm), so a checkpoint is ≈5 MB. Validation on val30 every 250 steps (every 500 on saved snapshots at 50 shots, where inference does not fit next to the training state); the best step is kept. Runs: FloodNet 1, 5, 10, 20, 50 shots and RescueNet 20, 50 (seed 0). **Inference:** fp16, 6 tiles, 17 s per photo at 50 shots (12 GB). **Decode tuning** on val30: coordinate ascent on per-channel τ with drivable-road precision not allowed to drop (a false "clear" sends a truck into water), then optional building-level rules (one flooded/dry label per building).

input	384 px crops (side 320–480 px)
augment	flips, rot90, bright./contrast 0.2
support (inference)	best 384 px window
batch; optimiser	2; Adam, lr 10^{-3} cosine, no decay
steps	1,000 (≤10, 50 shots), 2,000 (20)
precision	fp16 + scaler, grad. ckpt.
speed (T4)	2.6–4.1 s/step, 10–13.5 GB

Improvement round (FloodNet 50-shot only; 36 ideas, every one scored on val30 with its own re-tuned decode; rule fixed in advance: a free change is kept if val30 mIoU and drivable precision do not drop, a costly one needs +.010, and the switch needs a paired full-test gain with CI above 0). Kept: a **1,000-step** schedule with gradients only on the trained tensors (+.030 on val30; 30 % faster), the **weight average** of the step-750 and step-1,000 snapshots (+.002), a **guided filter** (edge-aware smoothing of the probabilities, guided by the photo) before the decode (+.001) and **SAM** prompted with each predicted building's box, which adds the building pixels Chameleon missed (+.002; a second model, adopted although below +.010). A clean-up step (+.003 mIoU) was removed again because it merged and deleted small houses (val30 building-detection F1 .680→.653). Everything else was dropped: e.g. training more parts (label decoder −.020, relative positions −.007), rare-class losses −.038, augmentations −.097, class-balanced supports −.106, pseudo-labels on 100 unlabelled photos −.070; denser tiles gained +.010 but double the prediction time and were cut for the GPU budget (full list: notebook §5.4).

Router (Dijkstra over (cell, mode) states; modes go truck → boat → on foot, never back into a vehicle). A truck drives on clear road at cost 1 per cell, plus 5 p_{wet} (risk-aware), or on dry land at 4×; on a *predicted* map only where $p_{\text{wet}} < g$ (**off-road gate**, $g = 0.3$ FloodNet, 0.2 RescueNet, each tuned on val30). A boat needs water or flooded road (3×, launch +40); the last stretch can be walked (8×, +10 to get out). Vehicles have a width (truck 2.5 m, boat 1.6 m; ground scale from the cars in the photo), parked cars do not block a road, boats pass under tree canopy, and nothing squeezes diagonally between two obstacles. The **labelled map** is the same router on the human labels: the datasets label pixels, not reachability, so this is our reference, not ground truth. We score the houses **cut off by road** (status ≠ TRUCK) on the predicted map against the labelled map (P, R, F1). **Confidence tiers:** each prediction is routed again with a cautious gate (0.05 FloodNet, 0.15 RescueNet, chosen on val30 as the largest gate whose truck-reachable houses are ≥99 % correct); a house a truck still reaches is *confident*, else *check first*. The **system**: a web viewer (Leaflet; the router ported to JavaScript and bit-exact with Python on all 200 scenes) and a Gradio app that runs the model live on an uploaded photo.

5 Results and analysis

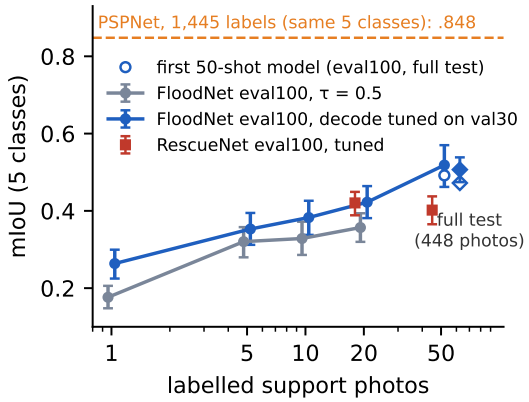


Figure 1: Learning curve (eval100, 95 % CIs over photos); diamonds: full test.

IoU (full test)	labels	r. clear	r. flood	b. flood	b. dry	water
ours, first model	50	.544	.291	.577	.500	.448
ours, final	50	**.662**	**.307**	**.594**	.498	**.472**
ENet [2]	1,445	.484	.125	.069	.474	.490
DeepLabV3+ [2]	1,445	.702	.520	.327	.728	.752
PSPNet [2]	1,445	.912	.822	.689	.898	.920

Table 1: Per class, full 448-photo test (tuned decodes). Baselines: FloodNet paper (10 classes, 713×713); our 5-channel decode avoids tree/grass confusions, which may favour us slightly.

Segmentation (Track A). The curve rises from .26 (1 photo) to .52 (50 photos, final model) on eval100 with the tuned decode (Fig. 1); paired bootstrap differences show real gains from 1→5 (+.14 [+.10, +.18], τ 0.5) and 20→50 (+.10 [+.05, +.14], tuned), but 5→10 is indistinguishable from zero (+.01 [−.03, +.05]). On the full test the final model reaches **.507** [.474, .538] (first model .472, paired +.035 [+.013, +.057]; Chameleon alone without SAM .503), above ENet on 4 of 5 classes and DeepLabV3+ on flooded buildings, far below PSPNet; **flooded roads are the gap** (.31 vs .82). The gain is mostly clear roads (.54→.66) and water; drivable precision rose .830→.845. Its decode matters: the shorter schedule shifted the calibration, so at a plain τ 0.5 it scores only .278; the per-channel thresholds chosen on

val30 are part of the model. The first model never learned a flooded/dry signal for buildings (per-building AUC ≈0.5); a rule tuned on val30 ("flooded if water lies within ∼10 m") fixes dry buildings at 5–20 shots. **Cross-disaster:** the FloodNet model applied to RescueNet with zero RescueNet labels reaches about half of what 20 RescueNet labels give (mean road/building/water IoU .32 vs .63): a new disaster needs its own few labels. RescueNet 20-shot: .421 (eval100, tuned); 50 shots did not help (.402). **Failure analysis** (16 worst eval100 photos, tagged by eye): the dangerous error is **shallow or muddy water decoded as dry land**; on the first model, 92 % of the labelled-water cells on predicted truck routes had been crossed as dry off-road land (median p_{wet} .21), which is why the off-road gate exists. Most other errors are benign for routing (water vs flooded road, both boat cells) or are label ambiguities (bridges labelled "flooded road").

Routing (Track B). Houses cut off by road, prediction vs labelled map: FloodNet full test (1,232 houses) P .827, R .690, **F1 .752** [.629, .851]; eval100 **.819** [.722, .893]; RescueNet 20-shot eval100 **.818** [.524, .966] (only 25 cut-off houses). The better map did *not* improve routing: vs the first model F1 changed by −.007 [−.048, +.026] (full test) and −.015 [−.060, +.020] (eval100), both within noise. The off-road gate lifted the first model's val30 F1 from .556 to .980; vehicle widths add .770→.819 (FloodNet eval100). Statuses do not depend on the cost values at all (14 cost changes, 0 status changes), and F1 stays within .80–.84 when the gate, cell size, goal ring or vehicle widths change. **Confidence tiers:** of 495 houses the prediction calls truck-reachable on FloodNet eval100, the 208 *confident* ones are **100 %** truly truck-reachable (cautious gate re-tuned for the final model), the 287 *check first* 79 %; in the honest-failure scene 7241 all 6 wrongly truck-reachable houses are *check first*. Buildings found on the predicted map match labelled buildings with F1 .81 (first model .85; RescueNet .63). Test-time flips (4 views, first model) were tried and **not adopted**: with their own val30-tuned decode the full test fell to .455.

System. Fig. 2 shows the router on a predicted and a labelled map. The notebook runs end to end on a fresh T4 in 26 minutes (Kaggle; 30 on Colab) and **recomputes the 50- and 20-shot models live: identical to the stored predictions on Kaggle**, where they were made (maximum difference 0; on Colab within 3/255, 99.99 % of pixels decoding the same), then routes an uploaded photo through the app in 16 s. The viewer shows all 200 eval100 photos, animated routes, minutes and metres, and the water probability behind each "check first".



Figure 2: Badges: house status (green truck, orange boat, violet on foot, yellow *check first*); route: white truck, cyan boat, violet dots on foot. Left/middle: the same flooded street, prediction vs labelled map; the selected house needs a boat on both. Right: RescueNet canal row, truck → boat across the canal → on foot.

Limitations. US hurricane suburbs are not Korean urban floods (dense blocks, underpasses invisible from above); single nadir frames without georeferencing or mosaics; the labelled map encodes our router's assumptions, not field-checked access; one seed per point; val30 (30 photos) is reused for checkpoints, decodes, gates and the 36 improvement ideas, so val30 numbers are optimistic (eval100 and the full test were not used for any choice; the switch needed a paired full-test gain); minutes and metres are rough estimates.

References

[1] D. Kim, S. Cho, S. Kim, C. Luo, S. Hong. *Chameleon: A Data-Efficient Generalist for Dense Visual Prediction in the Wild*. ECCV 2024. arXiv:2404.18459; https://github.com/GitGyun/chameleon.

[2] M. Rahnemoonfar, T. Chowdhury, A. Sarkar et al. *FloodNet: A High Resolution Aerial Imagery Dataset for Post Flood Scene Understanding*. IEEE Access 9, 2021. arXiv:2012.02951.

[3] M. Rahnemoonfar, T. Chowdhury, R. Murphy. *RescueNet: A High Resolution UAV Semantic Segmentation Dataset for Natural Disaster Damage Assessment*. Scientific Data 10, 2023. doi:10.1038/s41597-023-02799-4.

[4] A. Kirillov, E. Mintun, N. Ravi et al. *Segment Anything*. ICCV 2023. arXiv:2304.02643.

[5] Z. Peng, L. Dong, H. Bao et al. *BEiT v2: Masked Image Modeling with Vector-Quantized Visual Tokenizers*. arXiv:2208.06366, 2022.

[6] NPR, *Historic rainfall turns Seoul's roads into rivers and leaves 8 dead*, 9 Aug 2022. https://www.npr.org/2022/08/09/1116455746/historic-rainfall-turns-seouls-roads-into-rivers-and-leaves-8-dead.

[7] The Korea Times, *50 dead or missing in downpours after 5 more bodies recovered from underground road*, 18 Jul 2023. https://www.koreatimes.co.kr/www/nation/2023/07/281_355125.html.

Code, data bundle and every reported number: the notebook `20260876_MarlenMelis_code.ipynb` and `src/rebuild_all.py`.